

Flat Minima

Una lectura guiada del artículo clásico
de Hochreiter & Schmidhuber (1997)

NEURAL COMPUTATION 9(1): 1–42

*“Un mínimo plano es una región amplia en el espacio de pesos
donde muchas configuraciones llevan al mismo error pequeño.”*

Enfoque pedagógico:

Este apunte explica cada sección con lenguaje accesible. Cada concepto técnico complejo tiene una **definición** formal, un **ejemplo intuitivo** y una **nota técnica** breve. Pensado para estudiantes que se acercan por primera vez a la teoría de la generalización en redes neuronales.

Autor del apunte: Estudiante de IPRE (2026-2)

Fuente: FlatMinima1997.pdf

Documento elaborado para uso académico personal

Índice

Cómo leer este apunte	2
1. La idea central: ¿qué son los mínimos planos?	3
1.1. El problema: entrenar una red neuronal	3
1.2. Mínimos: puntos de bajo error	3
1.3. ¿Por qué los mínimos planos son mejores?	3
2. Sección 1 — Ideas básicas y esquema (Basic Ideas / Outline)	3
2.1. Qué propone el artículo	3
2.2. El esquema del artículo	4
3. Sección 2 — Tarea, arquitectura y cajas	4
3.1. La tarea de generalización	4
3.2. Las “cajas” en el espacio de pesos	4
3.3. Error tolerable y mínimos aceptables	5
3.4. Notación matemática simplificada	5
4. Sección 3 — El algoritmo FMS	5
4.1. La función objetivo que minimiza FMS	5
4.2. Cómo se calcula el gradiente	5
4.3. El papel del hiperparámetro λ	6
5. Sección 4 — Derivación formal	6
5.1. Condición de planitud 1: robustez a perturbaciones	6
5.2. Condición de planitud 2: igual planitud en todas las direcciones	6
5.3. Aproximación lineal	6
6. Sección 5 — Experimentos	7
6.1. Experimento 1: clasificación ruidosa	7
6.2. Experimentos 3–5: predicción del mercado bursátil alemán (DAX)	7
7. Sección 6 — Comparación con métodos previos	7
7.1. Categoría 1: priors sobre los pesos	7
7.2. Categoría 2: validación y poda	7
8. Sección 7 — Limitaciones y futuro	8
8.1. Limitaciones reconocidas por los autores	8
8.2. Aplicaciones futuras mencionadas	8
9. Apéndice — Justificación teórica (resumen pedagógico)	8
9.1. A.1 — Mínimos planos como hipótesis más probables	8
9.2. A.2 — ¿Por qué la Hessiana disminuye?	8
9.3. A.3 — Implementación eficiente	8
10. Glosario pedagógico	9
11. Reflexión y cierre	9

Cómo leer este apunte

Este documento acompaña la lectura del artículo original. Se recomienda tener a mano el PDF mientras se lee. Cada sección del apunte corresponde a una sección del artículo. Hay tres tipos de recuadros:

Definición 1: Ejemplo de uso

Un recuadro **Definición** explica un concepto técnico con palabras simples y luego con la notación matemática del artículo.

Ejemplo intuitivo 1: Ejemplo de uso

Un recuadro **Ejemplo intuitivo** ofrece una analogía o un caso concreto que no requiere saber matemáticas para entender la idea.

Nota técnica 1: Nota técnica

Un recuadro **Nota técnica** profundiza para quienes ya dominan algo del tema o quieren ver la conexión matemática precisa con el artículo original.

1. La idea central: ¿qué son los mínimos planos?

1.1. El problema: entrenar una red neuronal

Cuando entrenamos una red neuronal, tenemos datos de ejemplo (x_p, y_p) : una entrada x_p y una salida deseada y_p . La red tiene **pesos** w (números que conectan las unidades entre sí). El objetivo del entrenamiento es ajustar los pesos para que la red cometa poco error con los datos.

Definición 2: Pesos (w) y espacio de pesos (W)

Los **pesos** son los parámetros que la red aprende. Si la red tiene L conexiones, entonces $w = (w_1, w_2, \dots, w_L)$ es un vector con L números. El **espacio de pesos** $W \subset \mathbb{R}^L$ es el “mapa” donde cada punto representa una red posible.

1.2. Mínimos: puntos de bajo error

Imagina que dibujamos el error de la red como una montaña: el eje x y y son los pesos, y la altura es cuánto se equivoca la red. Queremos llegar al fondo del valle: un punto donde el error es mínimo. Eso es un **mínimo** de la función de error.

Ejemplo intuitivo 2: Mínimo agudo vs. mínimo plano

Mínimo agudo (sharp): es como el fondo de un pozo estrecho y profundo. Si te mueves un poco en cualquier dirección, subes mucho (el error aumenta rápido). Eso significa que los pesos deben ser muy precisos.

Mínimo plano (flat): es como un valle ancho y poco profundo. Puedes moverte bastante en varias direcciones sin subir mucho (el error se mantiene bajo). Eso significa que los pesos pueden ser imprecisos.

1.3. ¿Por qué los mínimos planos son mejores?

El artículo argumenta con la teoría **MDL (Minimum Description Length)**: un modelo simple es aquel que se puede describir con pocos bits. Un mínimo plano permite describir los pesos con menor precisión (menos bits), porque pequeños cambios no alteran la función. Por tanto, los mínimos planos corresponden a redes “simples” que generalizan mejor.

Nota técnica 2: Conexión con MDL y la teoría de la información

Según Rissanen (1978) y Wallace (1968), la longitud de descripción de un modelo está relacionada con su complejidad. Si los pesos de un mínimo pueden representarse con baja precisión (pocos bits por peso), la descripción total del modelo es corta. El artículo formaliza esto con una variante del algoritmo de Gibbs (ver Apéndice A.1).

Resumen en una frase: *No todos los mínimos son iguales. Los mínimos planos (valles anchos) corresponden a redes más simples que generalizan mejor.*

2. Sección 1 — Ideas básicas y esquema (Basic Ideas / Outline)

2.1. Qué propone el artículo

Los autores presentan un algoritmo llamado **FMS (Flat Minimum Search)** que busca mínimos planos en lugar de cualquier mínimo. A diferencia de métodos anteriores (como weight decay o “optimal brain surgeon”), FMS:

- No depende de elegir un “buen” prior sobre los pesos.
- Usa un prior sobre las **funciones de entrada/salida**, que automáticamente toma en cuenta la arquitectura de la red y el conjunto de entrenamiento.
- Requiere calcular derivadas de segundo orden (la Hessiana), pero con métodos eficientes mantiene la misma complejidad computacional que el backprop estándar.
- Automáticamente poda (elimina) unidades, pesos y líneas de entrada que no son importantes.

2.2. El esquema del artículo

El artículo se organiza así (extracto del propio artículo, traducido y simplificado):

1. **Sección 2:** Define los conceptos básicos: medidas de error, mínimos aceptables, cajas (M_w) en el espacio de pesos.
2. **Sección 3:** Presenta el algoritmo FMS.
3. **Sección 4:** Deriva formalmente el algoritmo (las condiciones de planitud).
4. **Sección 5:** Muestra resultados experimentales con redes feedforward y recurrentes.
5. **Sección 6:** Compara con trabajos previos.
6. **Sección 7:** Limitaciones y trabajo futuro.
7. **Apéndice:** Justificación teórica detallada (Gibbs, MDL, derivaciones de la Hessiana, pseudocódigo).

Definición 3: Flat Minimum Search (FMS)

FMS es el algoritmo propuesto. En lugar de minimizar solo el error de entrenamiento, minimiza una función que combina el error con una medida de la “planitud” de la región alrededor de los pesos actuales. Esto se hace calculando cuánto cambia la salida si perturbamos los pesos dentro de una “caja”.

3. Sección 2 — Tarea, arquitectura y cajas

3.1. La tarea de generalización

Queremos aproximar una función desconocida $f : X \rightarrow Y$, donde $X \subset \mathbb{R}^N$ (entradas) y $Y \subset \mathbb{R}^K$ (salidas). El conjunto de datos D se divide en:

- D_0 : datos de entrenamiento (usados para ajustar los pesos).
- $D \setminus D_0$: datos de prueba o test (usados para medir generalización, no para ajustar pesos).

La red neuronal es una función $net(w) : x \mapsto o(w; x)$, donde $o(w; x) \in \mathbb{R}^K$ es la salida de la red con pesos w y entrada x .

3.2. Las “cajas” en el espacio de pesos

Un concepto clave del artículo es la **caja** (box) M_w centrada en un vector de pesos w . Cada lado de la caja es paralelo a un eje del espacio de pesos. Si tenemos L pesos, la caja es un **hipercuboide** de L dimensiones.

Ejemplo intuitivo 3: Visualización de una caja en 2 dimensiones

Si la red solo tuviera 2 pesos (w_1 y w_2), la caja sería un rectángulo centrado en (w_1, w_2) . El ancho en la dirección de w_1 es $2\delta_1$, y en la de w_2 es $2\delta_2$. Un mínimo “plano” es aquel donde este rectángulo puede ser muy grande sin que el error suba por encima de un umbral E_{tol} .

3.3. Error tolerable y mínimos aceptables

Se define $E_{\text{tol}} > 0$: el error máximo “aceptable”. Un peso w es un **mínimo aceptable** si dentro de su caja M_w el error nunca supera E_{tol} y es aproximadamente constante (es decir, la caja es “plana”).

Definición 4: Mínimo aceptable

Un **mínimo aceptable** es un vector de pesos w tal que existe una caja M_w (centrada en w) dentro de la cual todas las redes inducidas por los pesos en la caja tienen un error de entrenamiento menor o igual a E_{tol} , y la salida de la red no cambia significativamente.

3.4. Notación matemática simplificada

- w_{ij} : peso de la conexión de la unidad j a la unidad i .
- $w \in \mathbb{R}^L$: vector que agrupa todos los w_{ij} .
- $o_k(w; x)$: activación de la unidad de salida k con pesos w y entrada x .
- $E(D_0; w)$: error cuadrático medio entre las salidas de la red y los objetivos y_p para los datos D_0 .

Nota técnica 3: Sobre el error cuadrático

En el artículo, el error cuadrático para un ejemplo (x_p, y_p) es $E(w; x_p) = \frac{1}{2} \sum_k (y_{pk} - o_k(w; x_p))^2$. El factor $1/2$ es convencional y simplifica las derivadas.

4. Sección 3 — El algoritmo FMS

4.1. La función objetivo que minimiza FMS

FMS no minimiza solo el error. Minimiza:

$$B(w; X_0) = C + \lambda \sum_{p \in X_0} B(w; x_p) \quad (1)$$

donde C es una constante de regularización, $\lambda > 0$ es un hiperparámetro que controla el equilibrio entre error y planitud, y $B(w; x_p)$ es un término adicional que mide cuánto cambia la salida de la red si perturbamos los pesos dentro de la caja.

4.2. Cómo se calcula el gradiente

Para minimizar B , necesitamos su gradiente (derivada primera) con respecto a cada peso w_{uv} . Esto requiere calcular derivadas segundas de la salida de la red ($\partial^2 o_k / \partial w_{ij} \partial w_{uv}$), que involucran la **Hessiana** de cada unidad de salida.

Nota técnica 4: Complejidad computacional

Aunque parece que calcular derivadas segundas debería ser mucho más lento que el backprop (que usa solo derivadas primeras), los autores usan los métodos eficientes de Pearlmutter (1994) y Müller (1993). Esto permite calcular el producto de la Hessiana por un vector en $O(L)$, es decir, con la misma complejidad que un paso de backprop.

4.3. El papel del hiperparámetro λ

λ controla el “compromiso” entre minimizar el error y maximizar la planitud. Un λ grande favorece mínimos muy planos (posiblemente con mayor error de entrenamiento). Un λ pequeño favorece mínimos con menor error (posiblemente más agudos).

Ejemplo intuitivo 4: Analogía del hiperparámetro

Imagina que buscas un lugar para acampar. Quieres que sea bajo (poco error) pero también que sea un terreno amplio (plano), porque si el terreno es estrecho y empinado, cualquier paso te haría caer (generalizar mal). λ es como tu preferencia personal: ¿prefieres un lugar muy bajo aunque sea pequeño, o un lugar un poco más alto pero con mucho espacio alrededor?

5. Sección 4 — Derivación formal

Esta sección es la más matemática del artículo. Explicamos las ideas clave sin entrar en todas las ecuaciones.

5.1. Condición de planitud 1: robustez a perturbaciones

Se define una medida $E_D(w; \tilde{w})$ que cuantifica cuánto cambia la salida si cambiamos los pesos de w a $\tilde{w}$ (donde $\tilde{w}$ está dentro de la caja). La condición exige que este cambio sea pequeño. Es una condición de “tolerancia a perturbaciones” o “robustez”.

5.2. Condición de planitud 2: igual planitud en todas las direcciones

No basta con que la caja sea grande en una sola dirección. Queremos que sea grande en todas. Esto se impone exigiendo que una expresión cuadrática (que involucra las derivadas segundas) sea igual en todas las direcciones. El resultado es que el algoritmo minimiza la **varianza** de las funciones inducidas por los pesos dentro de la caja.

5.3. Aproximación lineal

Para hacer los cálculos manejables, los autores aproximan los cambios con una expansión lineal (primer orden) y cuadrática (segundo orden). Esto es válido si la caja es suficientemente pequeña, y el artículo prueba en el Apéndice A.2 que, a medida que el algoritmo avanza, las cajas válidas se hacen más grandes.

Definición 5: Aproximación lineal y cuadrática

Una **aproximación lineal** usa solo la derivada primera para aproximar una función cerca de un punto. Una **aproximación cuadrática** usa también la derivada segunda, capturando la curvatura. Cerca de un mínimo, la aproximación cuadrática es mucho más precisa que la lineal.

6. Sección 5 — Experimentos

6.1. Experimento 1: clasificación ruidosa

Tarea: decidir si un punto (x, y) en el plano tiene $x > 0$ (clase 1) o $x \leq 0$ (clase 0). Hay ruido: los datos se generan con una distribución gaussiana y luego se agregan errores aleatorios. El error inherente mínimo es 9.27 %.

Resultado: FMS deja una sola unidad oculta activa (la que usa solo el eje x) y poda el resto. Backprop, en cambio, usa muchas unidades ocultas y aprende los outliers (sobreajusta). La tabla del artículo muestra que FMS logra errores mucho más cercanos al óptimo.

6.2. Experimentos 3–5: predicción del mercado bursátil alemán (DAX)

Se predice la tendencia del índice DAX usando indicadores fundamentales (interés, producción, sentimiento empresarial) y luego técnicos (RSI, MACD, etc.).

Lección importante: El artículo señala que el MSE (error cuadrático medio) no es una buena medida para este problema. FMS a veces tiene mayor MSE que weight decay, pero hace más predicciones correctas. Esto ocurre porque FMS hace predicciones “fuertes” (valores grandes cuando está seguro), mientras que weight decay hace predicciones “tímidas” (valores cercanos a cero).

Ejemplo intuitivo 5: Por qué MSE puede ser engañoso

Si el objetivo real es +1 y predices +0,9, el error cuadrático es $(0,1)^2 = 0,01$. Si predices -0,9, el error es $(1,9)^2 = 3,61$. Weight decay podría predecir valores cercanos a 0 para evitar errores grandes, logrando un MSE bajo, pero haciendo pocas predicciones útiles. FMS se arriesga más y, aunque cometa errores grandes ocasionales, acierta más a menudo en la dirección correcta.

7. Sección 6 — Comparación con métodos previos

El artículo clasifica los métodos anteriores en dos categorías:

7.1. Categoría 1: priors sobre los pesos

Métodos como **weight decay** (Weigend et al., 1991; Krogh & Hertz, 1992) y **Hinton & van Camp** (1993) asumen que hay una distribución “buena” de los pesos (por ejemplo, gaussiana). El problema: no siempre existe un buen prior para todas las arquitecturas y datos.

Definición 6: Weight decay

Weight decay es una técnica de regularización que penaliza los pesos grandes sumando un término $\lambda \|w\|^2$ al error de entrenamiento. Esto favorece pesos pequeños, lo que puede mejorar la generalización, pero no distingue entre pesos importantes y no importantes de forma inteligente.

7.2. Categoría 2: validación y poda

Métodos como **optimal brain damage/surgeon** (LeCun et al., 1990; Hassibi & Stork, 1993) y los basados en **validación cruzada** (Stone, 1974) seleccionan la arquitectura antes o después del entrenamiento. No la ajustan durante.

La ventaja de FMS: Usa un prior sobre **funciones de entrada/salida**, no sobre los pesos. Esto toma automáticamente en cuenta la arquitectura de la red y el conjunto de datos, sin requerir que el usuario elija un buen prior de pesos.

8. Sección 7 — Limitaciones y futuro

8.1. Limitaciones reconocidas por los autores

1. **Ajuste de λ :** Se hace con una heurística (de Weigend et al., 1991). Una justificación teórica completa requeriría calcular derivadas de tercer orden, lo cual es impracticable.
2. **Cajas alineadas con ejes:** Las cajas M_w son hipercuboides con lados paralelos a los ejes. Esto puede subestimar el volumen real del mínimo plano. Sería interesante usar aproximaciones no alineadas.
3. **Múltiples inicializaciones:** El artículo sugiere que usar un “comité” de FMS con diferentes inicializaciones podría mejorar los resultados, pero esto queda como trabajo futuro.

8.2. Aplicaciones futuras mencionadas

Los autores sugieren que FMS podría ser útil para **aprendizaje no supervisado**, entrenando auto-codificadores (auto-associators) para obtener códigos eficientes (factoriales, locales o dispersos) de los datos. Experimentos iniciales con reconocimiento de vocales muestran resultados prometedores.

9. Apéndice — Justificación teórica (resumen pedagógico)

9.1. A.1 — Mínimos planos como hipótesis más probables

El artículo introduce una **nueva definición de error de generalización** que se divide en:

- **Error de subajuste (underfitting):** La hipótesis no aproxima bien los datos de entrenamiento (error grande en D_0).
- **Error de sobreajuste (overfitting):** La hipótesis se ajusta demasiado a D_0 y no funciona bien con datos nuevos.

Usando una variante del **algoritmo de Gibbs** y la **distancia de Kullback-Leibler** (que mide cuánto difiere una distribución de probabilidad de otra), los autores derivan que, entre todas las hipótesis con bajo subajuste, las que minimizan el sobreajuste relativo son las que corresponden a mínimos planos.

9.2. A.2 — ¿Por qué la Hessiana disminuye?

Se demuestra matemáticamente que en mínimos aceptables, los términos de la Hessiana correspondientes a pesos grandes tienden a ser pequeños (porque los pesos grandes hacen que la función sea robusta), mientras que los correspondientes a pesos pequeños pueden ser grandes. Esto confirma que FMS hace que los pesos “importantes” sean robustos.

9.3. A.3 — Implementación eficiente

Se describe cómo calcular el producto de un vector con la Hessiana de la salida en $O(L)$ usando los algoritmos de Pearlmutter y Müller. Esto hace que FMS sea computacionalmente viable, con la misma complejidad que un paso de backprop estándar.

10. Glosario pedagógico

Glosario de términos del artículo

Backpropagation (BP): Algoritmo que entrena redes neuronales propagando el error hacia atrás y ajustando los pesos según el gradiente.

Flat Minimum (mínimo plano): Región amplia en el espacio de pesos donde el error se mantiene bajo. Corresponde a redes simples con buena generalización.

Sharp Minimum (mínimo agudo): Región estrecha donde los pesos deben ser muy precisos. Corresponde a redes complejas con riesgo de sobreajuste.

MDL (Minimum Description Length): Principio de la teoría de la información que dice que el mejor modelo es el que describe los datos con menos bits.

Gibbs Algorithm: Método estadístico para muestrear distribuciones de probabilidad, usado aquí para modelar la distribución posterior sobre hipótesis.

Kullback-Leibler Distance (distancia K-L): Medida de cuánto difiere una distribución de probabilidad de otra.

Weight Decay (WD): Técnica de regularización que penaliza pesos grandes.

Optimal Brain Surgeon / Damage (OBS / OBD): Métodos de poda que eliminan pesos basándose en derivadas de segundo orden.

Overfitting (sobreajuste): El modelo memoriza los datos de entrenamiento (incluido el ruido) y no generaliza.

Underfitting (subajuste): El modelo es demasiado simple y no captura la señal de los datos.

Hessiana (H): Matriz cuadrada de derivadas segundas. Describe la curvatura de la función de error en cada dirección.

Eigenvalues (valores propios): Números asociados a la Hessiana que indican cuánto se curva la función en direcciones específicas. Valores pequeños = región plana.

11. Reflexión y cierre

Este artículo conecta tres áreas aparentemente distintas:

1. **Geometría del aprendizaje:** La forma de los mínimos (planos vs. agudos) en el espacio de pesos determina la calidad del modelo aprendido.
2. **Teoría de la información:** La longitud mínima de descripción (MDL) proporciona un criterio objetivo de simplicidad.
3. **Generalización:** Un modelo que puede ser descrito con pocos bits (porque sus pesos son robustos a perturbaciones) generaliza mejor.

La idea de que “simple es mejor” es una constante en el aprendizaje automático moderno, desde la regularización L_1 y L_2 hasta la teoría de la información aplicada al aprendizaje profundo. Este artículo de 1997 es un precursor fundamental.

Apunte pedagógico elaborado para uso personal — IPRE 2026-2

*Basado en: Hochreiter, S. & Schmidhuber, J. (1997). **Flat Minima**. Neural Computation, 9(1), 1–42.*